

LabTrack: Lab Reservation, Scheduling and Resource Management

One project for two courses: Operating Systems and Database Management Systems

Structure of this document: **Problem** → **How the system is used (flows)** → **Solution** → **OS features** → **DBMS features** → **How the two stay in sync** → **Course coverage and demos**

1. The Problem

1.1 Situation

A college computer lab has a fixed number of PCs, and demand is not steady. It spikes before assignment deadlines, during project weeks and around lab exams. Students come from hostels, departments and other buildings. Today the lab runs on a physical logbook and first-come-at-the-door access.

1.2 What goes wrong

Failure	Who feels it
A hostel student walks 15 minutes to the lab and finds every PC taken. The trip is wasted.	Students
Students log in, leave for tea or lunch, and the PC sits idle but "occupied".	Students, lab
Access is unfair: whoever arrives first wins, and a student with a deadline tonight gets the same treatment as someone browsing.	Students
Some work needs specific software (MATLAB, Vivado, AutoCAD). Licenses are limited and only some PCs have them.	Students, admin
Group projects need several PCs together, and nobody can plan for that.	Student teams
Logbook entries are wrong, missing an exit time, or faked.	Admin
Nobody can see which PCs are in use right now. A crashed PC still looks "occupied".	Admin
One student runs games, torrents or a heavy job and slows the machine for everyone after them.	Everyone
The admin has no data to answer "how many PCs do we need?" or "when is it busiest?"	Admin, management

1.3 Root causes

1. **Demand is invisible.** The lab only learns about a student when they arrive.
2. **Allocation happens at the door**, with no coordination and no priorities.
3. **Nothing controls how a PC is used** after it is handed out.
4. **There is no reliable, shared, durable record** of who used what and when.

1.4 Problem statement

Design a system that lets students and faculty **request lab computers in advance (including from outside the lab)**, **allocates scarce machines fairly and efficiently**, **controls and monitors each machine while it is in use**, and **keeps a consistent, durable, queryable record** of everything that happens, even with many users acting at once and machines failing at any time.

1.5 Why this is an OS problem and a DBMS problem at the same time

- Allocating scarce resources (PCs, license seats, time) among competing requesters, controlling their use, and surviving failures is exactly what an **operating system** does, at the scale of a lab instead of a single machine.
- All the state of that allocation (requests, reservations, sessions, usage) must be **shared by many concurrent users, consistent, durable and queryable**. That is what a **database system** does.

Neither half works alone: the scheduler's decisions are only trustworthy if they are transactions, and the database is only useful if something real enforces and reports what happens on the machines.

2. Where Scheduling Comes From

If students only walked in, logged in and used a free PC, there would be nothing to schedule: it would be a logbook with a login. Scheduling becomes necessary because of **three facts**:

1. **Requests arrive before use.** A student in the hostel asks for a PC for 4 pm. The system must decide now, among many requests, who gets which PC and when.
2. **Demand exceeds capacity at peak times.** Some requests must wait, be moved to another slot, or be prioritized, so a policy decides.
3. **A reserved PC is held for someone who is not there yet.** The hold must expire if they don't show up, or the lab wastes capacity.

Everything else in the project follows from these three facts. Walk-in students still exist and are handled as a special case of the same system (flow B).

3. Actors

Actor	What they do
Student	Requests a PC from anywhere (hostel, classroom), walks to the lab, checks in at the PC, works, logs out.
Faculty	Reserves PCs in bulk for a class, exam or project demo, with higher priority.
Lab administrator	Watches live status, sets policies, handles maintenance, searches history, reads reports.
Super admin	Manages labs, PCs, software, users, scheduling configuration.
The machines (agents)	Each lab PC runs a LabTrack agent that is the system's hands and eyes on the machine.

4. How the System Is Used: Flows

4.1 Flow A: Planned use from the hostel (the main flow)

#	Student / Admin	What the system does
1	Student opens the portal from the hostel and logs in with college credentials.	Authenticates; shows lab availability for today and tomorrow.
2	Student requests: "Tomorrow, 4-6 pm, needs MATLAB, assignment due tomorrow night."	Records the request with its priority class, duration, required software and deadline.
3	(Waits a moment.)	Scheduler looks for a PC that has MATLAB, is free for that window, and has a license seat available. Found: confirmed . Not found: waitlisted , with a position.
4	Student receives: "LAB1-PC-07 held for you. Arrive by 4:15 pm or the hold is released."	Starts a grace timer that begins at slot start. PC-07 is marked held for that student from 4:00.
5	Student walks to the lab. Other students see PC-07's lock screen: "Reserved until 4:15."	The agent on PC-07 keeps it locked and shows who it is reserved for.
6	Student sits down and logs in at PC-07 with the same credentials.	The agent checks identity against the reservation. Match: check-in. The reservation becomes an active session , a license seat is taken, resource limits are applied, and the entry time is recorded.
7	Student works.	Agent monitors processes and CPU/memory, enforces policy (blacklisted apps), watches idle time, and warns before time runs out.
8	At 5:55 pm: "5 minutes left. Save your work."	Quantum timer fires. If nobody is waiting, extension is offered; if the next holder is waiting, no extension.
9	Student logs out (or time ends).	Agent terminates leftover processes, wipes the temporary workspace, releases the license seat, and reports the exit time. The PC becomes available, or moves straight to held for the next person.
10	Admin sees the whole story on the dashboard and in history.	Duration is computed and the usage record is complete.

4.2 Flow B: Walk-in student

A student arrives without a reservation and logs in at a free PC.

- The system checks that PC's **next reservation**. If the walk-in's session can finish before it, the walk-in is allowed (this is **backfilling**: using the gap before a reservation).
- If not, the system says "This PC is reserved from 4:30. Try PC-12 or take 25 minutes here," and the student chooses.
- Walk-ins get the lowest priority class and never displace a confirmed reservation.

4.3 Flow C: No-show

The student does not arrive within the grace period.

- Hold expires. The reservation becomes **no-show**. The PC is released.
- The scheduler immediately re-runs and promotes the first suitable **waitlisted** request. That student is notified.
- The no-show counts against the first student's **reliability score**, which lowers their priority for future requests (fairness feedback).

4.4 Flow D: Deadline rush (scarcity)

Fifty students want PCs between 6 and 9 pm, and only 30 PCs have the needed software.

- The scheduler applies the active policy: students with the nearest deadlines go first, shorter requests may fit into gaps, and nobody waits forever (aging).
- Students who don't get a slot see their waitlist position and are offered alternatives ("a seat is free at 8 pm").
- Admin can switch policy live, and the effect is visible on the schedule.

4.5 Flow E: Faculty class or exam block

A faculty member reserves 40 PCs for Thursday 10–12 for a lab exam.

- The block is guaranteed. Individual student reservations that conflict are **rescheduled** (not silently dropped) and the students are notified.
- PCs are locked down for exam mode (restricted app list, no external network, if enabled).

4.6 Flow F: A PC fails

A PC stops responding (power cut, crash).

- The watchdog notices missing heartbeats and marks it **unreachable**.
- An active session on it is closed as **crashed** and the license seat is freed.
- Any upcoming reservation on that PC is **moved** to a similar free PC and the student is told, so they are not turned away at the door.

4.7 Flow G: Overstay and extension

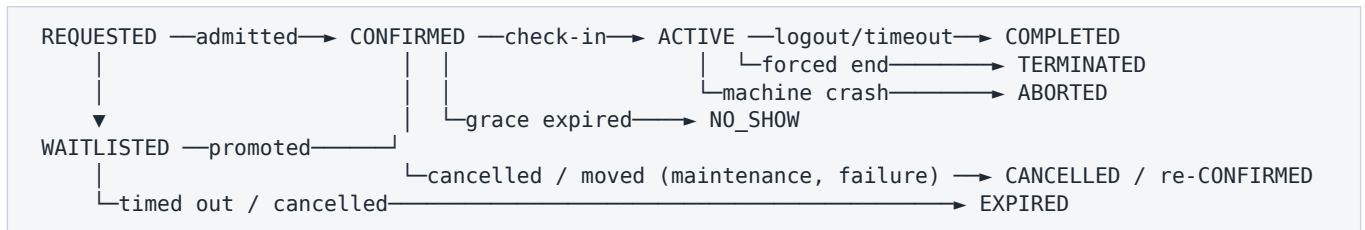
- A student asks for 30 more minutes. It is **granted** if no later reservation or waitlist demand conflicts, and **denied** otherwise.
- If a student ignores the end-of-time warnings, the agent logs them out in stages (warning → polite close request → forced termination).

4.8 Flow H: Admin's day

- Opens the dashboard: live PC states, a timeline of reservations per PC, queue length, violations, license usage.
- Marks a PC for maintenance: affected reservations are moved automatically.
- Searches history by student, PC or date. Reads reports (utilization, peak hours, no-show rate, department usage).
- Changes policies (blacklist, daily hour limit per student, scheduling algorithm, grace period).

4.9 State machines

Reservation lifecycle (deliberately modeled on a process lifecycle):

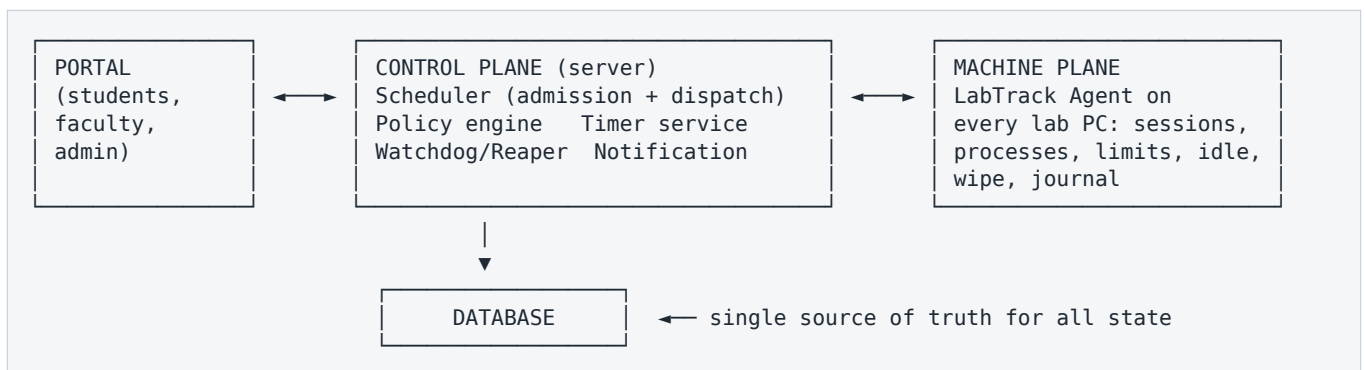


Reservation state	Process analogy
Requested	New
Waitlisted	Ready (waiting for the CPU)
Confirmed	Scheduled, dispatch pending
Active	Running
Completed / Terminated / Aborted	Exit / killed / crashed

PC lifecycle: AVAILABLE → HELD (reserved, awaiting check-in) → OCCUPIED → (cleanup) → AVAILABLE. From any state it can go to UNREACHABLE or MAINTENANCE.

5. The Solution (Overview)

5.1 Three planes



- **Portal:** where requests are made and status is seen (works from the hostel).
- **Control plane:** decides. The scheduler, the policy engine, timers and the watchdog all run here, and every decision is a database transaction.
- **Machine plane:** executes and reports. The agent on each PC enforces the decision on the real OS and reports what actually happened.

5.2 Design rules that keep OS and DBMS in sync

1. **The database is the single source of truth.** The scheduler and agents never keep authoritative state of their own. If it isn't committed, it didn't happen.
2. **Every state change is a transaction.** "Confirm reservation," "check in," "release PC" each succeed completely or not at all.
3. **The agent acts on committed state and reports back.** Commands go down, facts come up, and the database reconciles them.
4. **Time is a first-class concept.** Grace periods, time quanta, idle timeouts and slot boundaries are driven by timers on the OS side and by time-ranged data on the database side.
5. **History feeds the policy.** Recorded usage (no-show rates, peak hours, session lengths) tunes the scheduler.

5.3 The two-level scheduler (an OS idea applied to the lab)

Level	When it runs	What it decides	OS analogy
Admission scheduler	When a request arrives or capacity changes	Confirm now, waitlist, or reject. Which PC and slot.	Long-term (job) scheduler
Dispatcher	At slot start, check-in, no-show, session end	Hands a PC to the next holder: sets limits, locks or unlocks, applies cleanup.	Short-term scheduler + dispatcher / context switch

Policies (selectable at runtime and comparable in the simulator): FCFS, Shortest-Job-First (by requested duration), Priority with aging (exam > faculty > project > casual), Earliest-Deadline-First (assignment deadlines), Round Robin with time quantum (long sessions yield to waiting students), a multilevel queue with feedback (reliability score moves students between queues), and backfilling for walk-ins.

6. The Solution, Step by Step: Both Layers Together

Each step of the main flow, with the OS work and the DBMS work **at the same moment**.

Step	OS side	DBMS side
Request arrives	Portal's request handler is concurrent: many requests at once go onto a bounded request queue consumed by scheduler worker threads (producer-consumer).	Request stored with validity checks. Student's daily/weekly quota checked by a query. Reliability score and priority class looked up.
Scheduler picks a PC	Scheduler applies the policy (priority, EDF, SJF...). For group requests it searches for contiguous PCs (first-fit / best-fit) and checks for fragmentation.	A single transaction checks availability (including time overlap), checks license seats, locks the chosen PC and slot, and inserts the confirmation. Overlap and double booking are rejected by integrity rules, not just by code.
Hold and grace timer	A timer service (a min-heap of deadlines) fires at slot start + grace. The agent shows the reserved lock screen.	The hold is stored with its valid time range. Expiry is an atomic state change. A trigger records the PC's status history.
Student logs in at the PC	PAM-based login hook triggers the agent over a Unix domain socket . Privileges are separated: the UI is unprivileged, the agent daemon is privileged.	Credential check against hashed passwords. Check-in procedure: verifies the reservation, flips Reservation→Active, creates the session, takes a license seat, all in one transaction.
Session running	Agent threads sample /proc (CPU, RSS, page faults, process states), enforce cgroup limits , apply the blacklist (SIGTERM, then SIGKILL), run the idle timer.	Time-series samples are written to a partitioned table. Violations are inserted and a trigger updates the student's record. Row-level security shows each student only their own data.
Extension request	Quantum timer logic.	Query checks for conflicting later reservations and waitlist demand, then updates the end time under a lock.
Logout / expiry	Dispatcher kills leftover processes (reaping orphans), wipes the workspace, resets limits (the "context switch").	Close-session procedure: exit time, duration, license seat release, PC status, and promotion of the next waitlisted request, all atomic. Audit trigger fires.
No-show	Grace timer fires, agent unlocks the PC.	Reservation set to no-show, reliability score updated, next waitlisted request promoted using a skip-locked queue pop so parallel schedulers never take the same request.
PC crash	Watchdog timer misses heartbeats. Agent on restart reconciles its local journal with the server.	Orphaned session closed as crashed. Affected reservations re-allocated in one transaction. Recovery is based on the log, so nothing is lost.
Admin searches and reports	Dashboard reads while updates are happening (readers-writers problem).	Views, indexes, materialized views, window queries, and an analytics (warehouse) layer. Consistent snapshots under MVCC.

7. OS Features

Each feature is real, not simulated, unless marked. Cover them in this order of importance.

#	Topic	In LabTrack	Demonstrate
01	Process management	Agent uses system calls (fork , exec , kill , waitpid) and /proc . Tracks each student's process tree, process states, parent/child, zombie and orphan processes. Staged termination (SIGTERM → grace → SIGKILL).	Student starts a process tree; logout kills it cleanly; a zombie is created and reaped.
02	Threads and daemons	Agent is a systemd service with several threads: monitor, heartbeat, IPC listener, timer. Server uses a worker thread pool. Shared state protected by locks.	Show thread interaction, then remove a lock and show the corruption.
03	CPU-scheduling concepts at lab level	Two-level scheduler (admission + dispatcher). FCFS, SJF, Priority with aging, EDF, Round Robin quanta, multilevel feedback queue, backfilling. Cooperative preemption (warn, then terminate).	Same workload under each policy. Output: Gantt chart of PCs over time with waiting time, turnaround, response time, throughput, utilization, Jain's fairness index.
04	Synchronization	Mutex for PC assignment. Counting semaphore for license seats. Bounded buffer for telemetry. Readers-writers for dashboard vs updates. Condition variables to wake the scheduler.	Race demo: 50 simultaneous requests for one PC, with and without protection.
05	Deadlock	Group request needs several PCs plus license seats. Two groups each holding part of what the other needs is a deadlock. Prevention via all-or-nothing allocation and global resource ordering. Avoidance via a Banker's-style safety check for license seats. Detection via wait-for graph. Recovery by victim selection (lowest priority).	Create the deadlock, show detection, then show prevention making it impossible.
06	Memory management	(a) Allocating groups of PCs across space and time is contiguous allocation (first-fit, best-fit, worst-fit) with external fragmentation and compaction (moving reservations). (b) Per-session RSS tracking and cgroup memory limits with OOM behavior. (c) Page-fault counts from /proc used to flag thrashing machines.	Fragmentation chart per allocation strategy; a memory hog hits its cgroup limit; a thrashing PC is flagged.
07	Storage and file systems	Per-session workspace with disk quota, strict permissions, wipe on logout. Disk usage monitored per user.	Fill a quota and show enforcement; show the wipe.
08	IPC and networking	Unix domain socket and message protocol between login hook, UI and agent. TCP/TLS between agent and server. WebSocket for push commands. Signals as IPC.	Trace messages for one full login.
09	Timers and interrupt-style events	Grace timer, quantum timer, idle timer, heartbeat timer, all driven by an event loop and a timer queue .	Show each timer firing and the action it triggers.
010	Protection and security	UID/GID, file permissions, least-privilege agent, privilege separation, PAM authentication, process isolation via cgroups and namespaces. Blacklist and quota policies. Per-machine authentication tokens.	Attempt privilege escalation from the student side and show it blocked.
011	Resource limits and containers	Limits per priority class (CPU share, memory, process count). Optional: each session in its own namespace or container (stretch).	Two students on one machine's resources: one is capped, one is not.
012	Fault tolerance and recovery	Watchdog, reaper for orphaned sessions, agent local journal (write-ahead idea), checkpoint and restart, idempotent replay.	Kill the agent and the network mid-session. Show full recovery with no lost events.
013	Boot and init	Agent starts at boot, reconciles with the server (recovery on startup), enters a safe state if the server is unreachable.	Reboot a lab VM during a session.

8. DBMS Features

Each item is tied to a real need in the flows.

#	Topic	In LabTrack	Demonstrate
D1	Conceptual modeling (ER / EER)	Full ER model: strong and weak entities, specialization (Student / Faculty / Admin as kinds of User), a ternary relationship (Reservation ↔ Student ↔ PC ↔ Software), aggregation, cardinality and participation constraints, derived attributes (duration, waitlist position).	The ER diagram, plus the reasoning for each modeling choice.
D2	Relational design and normalization	Mapping ER to relations. Candidate, primary, foreign, composite and surrogate keys. Functional dependencies worked out for each relation, normalization to 3NF/BCNF, and a <i>justified denormalization</i> for high-volume monitoring data.	Show a bad unnormalized design with its insert/update/delete anomalies, then the fixed one.
D3	Integrity constraints	Domain, key, referential (with ON DELETE/UPDATE actions), CHECK constraints, and business rules enforced by the database : no double-booking of a PC for overlapping time, one active session per student, license seats never negative.	Try to break each rule from the application and show the database refusing.
D4	Temporal data	Time ranges for reservations (valid time) and separate status history (what the system believed when). Overlap detection, "what was free at 4 pm last Tuesday?" queries.	Time-travel queries over past states.
D5	Relational algebra	Key queries (availability, conflicts, history) written first in relational algebra, then in SQL.	The algebra for 5 core queries and its SQL equivalent.
D6	SQL, wide coverage	Joins (inner, outer, self, anti), correlated subqueries, set operations, grouping and HAVING, window functions (ranking, running totals, peak detection), recursive CTEs , views, updatable views, text search for names.	A query set with each feature mapped to a real report.
D7	Procedural database logic	Stored procedures for admission, check-in, close-session, promotion. Triggers (row and statement level) for audit, status history, violation counters, reliability score. Cursors. Exception handling.	Show the procedure rolling back cleanly on a conflict.
D8	Transactions and ACID	Every state change in section 6 is a transaction.	Atomicity: kill the process mid-transaction and show no partial state. Durability: crash and restart.
D9	Isolation levels and anomalies	"Two students book the last PC" is a textbook write skew / phantom case. Reproduce dirty read, non-repeatable read, phantom, lost update, write skew at each isolation level.	A table of anomaly vs isolation level, each shown live.
D10	Concurrency control mechanisms	Row locks, FOR UPDATE , skip-locked queue pop for parallel schedulers, optimistic concurrency (version check) for editing reservations, advisory locks. MVCC compared with two-phase locking. Database deadlock from opposite lock order, with detection and the lock-ordering fix.	Throughput and conflict-rate comparison across strategies under 100 concurrent clients.
D11	Recovery	Write-ahead logging, checkpoints, crash recovery, backups, point-in-time recovery .	Kill the database mid-load, restart, and verify. Restore to "10 minutes ago".
D12	Indexing and storage	B+tree, hash, range-type (GiST) index for time overlap, BRIN for time-series, partial and covering indexes, partitioning, clustering, buffer cache.	Run queries on a million synthetic sessions with and without each index and compare.
D13	Query processing and optimization	Read query plans and cost estimates. Compare nested-loop, hash and merge joins. Statistics and their effect on plans. Rewrite a slow query.	Before/after plans for three slow queries, with times.
D14	Security	Roles and privileges (least privilege), row-level security (students see only their own reservations), hashed passwords, SQL-injection prevention with parameterized queries, append-only audit log.	An injection attempt blocked. A student attempting to read another's data.
D15	Data warehousing and OLAP	An analytical layer separate from the live database: a star-style design (usage facts by time, PC, department, software), ETL jobs, rollup and cube queries, materialized views.	"Hours used by department by week," drill-down and roll-up on a dashboard.

#	Topic	In LabTrack	Demonstrate
D16	Replication and distribution (stretch)	A read replica serves the dashboard and reports so they never slow reservations. CAP and consistency trade-offs discussed with the real setup.	Replica lag shown live.
D17	Analytics that drive decisions	No-show rate by hour, demand forecast, and overbooking policy chosen from history.	The scheduler improves measurably after using the learned rates.

9. How the Two Courses Stay in Sync

9.1 Same problem, two lenses

Problem in LabTrack	OS lens	DBMS lens
Two students want the last PC	Mutual exclusion, semaphores	Isolation level, locks, write skew
Group needs PCs plus licenses	Deadlock, resource-allocation graph	Lock-wait deadlock, lock ordering, atomic transaction
System crashes mid-session	Watchdog, journal, restart	WAL, crash recovery, PITR
Queue of waiting students	Scheduling algorithm	Queue ordering with indexes and skip-locked
Student exceeds allowed usage	cgroup limit, signals	Constraints, quota checks, triggers
Who may see or do what	Permissions, UID, privilege separation	Roles, row-level security
Things that must happen at a time	Timers, event loop	Time ranges, scheduled jobs
What happened on the machine	<code>/proc</code> , process states	Time-series storage, partitions, OLAP

9.2 Feedback loops (the system improves itself)

1. **OS events change DB state, and DB state drives OS actions.** A no-show on the machine lowers the student's score in the database, which changes future scheduling.
2. **Database history tunes the scheduler.** No-show rates and peak hours decide overbooking and policy choice.
3. **The simulator replays real database traces** under each scheduling policy before an admin switches policy.
4. **Telemetry from agents flows into the same database that the dashboard and reports read**, so there is one version of the truth.

10. Course Coverage Checklists

10.1 For the OS course

- ☐ System calls and process control (O1)
- ☐ Threads, daemons (O2)
- ☐ CPU scheduling: FCFS, SJF, Priority, RR, EDF, MLFQ, with metrics and Gantt charts (O3)
- ☐ Synchronization: mutex, semaphore, producer-consumer, readers-writers (O4)
- ☐ Deadlock: prevention, avoidance, detection, recovery (O5)
- ☐ Memory management: allocation strategies, fragmentation, limits, page faults (O6)
- ☐ File systems and storage: quotas, permissions (O7)
- ☐ IPC and networking (O8)
- ☐ Timers and events (O9)
- ☐ Protection and security (O10)
- ☐ Containers and resource control (O11)
- ☐ Fault tolerance (O12) and boot/init (O13)

10.2 For the DBMS course

- ☐ ER and EER modeling (D1)
- ☐ Relational design, keys, FDs, normalization (D2)

- ☐ Integrity constraints (D3), temporal data (D4)
 - ☐ Relational algebra (D5), SQL (D6)
 - ☐ Stored procedures, triggers (D7)
 - ☐ Transactions, ACID, isolation, anomalies (D8, D9)
 - ☐ Concurrency control, MVCC vs 2PL, deadlocks (D10)
 - ☐ Recovery and backup (D11)
 - ☐ Indexing, storage, query optimization (D12, D13)
 - ☐ Security (D14)
 - ☐ Data warehousing and OLAP (D15), replication (D16), analytics (D17)
-

11. Demonstrations

For the OS evaluation

1. Full flow: reserve, lock screen, check-in, monitored session, forced logout, cleanup.
2. Scheduling policies compared on one workload, with Gantt charts and metrics.
3. Race condition for the last PC, with and without synchronization.
4. Deadlock created, detected, then prevented.
5. cgroup limits stop a memory hog; a blacklisted process is warned and killed.
6. Kill the agent and the network mid-session and show recovery.

For the DBMS evaluation

1. ER diagram, normalization walk-through with anomalies.
2. The database refusing every rule violation (double booking, overlaps, negative seats).
3. Isolation anomalies shown level by level, with the last-PC write-skew case.
4. Database deadlock and its resolution; skip-locked scheduler workers.
5. Index and query-plan comparisons on a million rows.
6. Crash the database mid-transaction, then point-in-time recovery.
7. Row-level security and injection defense; OLAP drill-down.

Joint

1. A 100-user simulated deadline rush with waitlist, no-shows, a PC failure, and an admin policy switch, ending with the dashboard and OLAP report from the same run.
-

12. Build Order and Minimum Version

1. **Foundation:** ER model, normalized design, core transactions for reserve / check-in / close. Portal login and basic dashboard.
2. **Machine plane:** agent, check-in, process sampling, idle and end-of-time handling, cleanup.
3. **Scheduling:** reservation, grace timers, waitlist, FCFS and Priority first, then the other policies.
4. **Control and safety:** policies, cgroup limits, licenses and semaphores, group requests and deadlock handling.
5. **Reliability:** watchdog, journal, re-allocation after failures.
6. **Depth work for each course:** isolation and concurrency experiments, indexing and query plans, recovery drills, simulator and OLAP layer.

Minimum version that still counts for both courses: hostel reservation with grace timer, check-in at the PC, monitored sessions with limits and cleanup, three scheduling policies with the simulator, license semaphore, the normalized design with constraints, procedures and triggers, the isolation and concurrency experiments, and crash recovery.

13. Practical Deployment Notes

- **Privacy:** only process names and resource counters are recorded, never screens, keystrokes or file contents. Students are told this at login.
- **Safe development:** build and test the agent in virtual machines. Never test kill policies or login changes on your own computer.
- **Rollout:** run in "observe only" mode for a week first (record, don't enforce), then switch on enforcement.
- **Fair use of power:** forced termination always follows warnings, and every forced action is recorded and explainable.